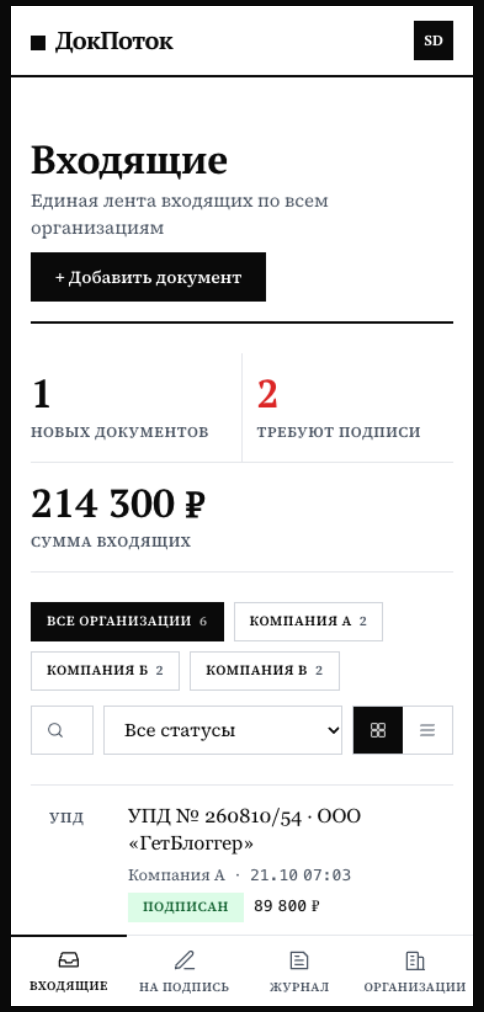
ДокПоток

Единая входящая ЭДО для группы компаний



Проектная работа курса «Разработка с AI-агентами»
Анастасия Топчий · октябрь 2026

topchiiaa-eng.github.io/docflow · github.com/topchiiaa-eng/docflow

Проблема: один менеджер — N ящиков ЭДО

20–30

минут в день уходит на обход ящиков трёх юрлиц в Контур.Диадоке (замер на собственной практике)

дни

задержка, с которой замечают документ «требуется подпись» — и срыв сроков документооборота

Кто страдает

Операционные менеджеры и бухгалтеры малых групп компаний (2–10 юрлиц): один человек ведёт документооборот нескольких ООО

Почему так

Интерфейсы ЭДО рассчитаны на «одна организация — один ящик»; несколько юрлиц = переключение контекста

Что было до проекта

Telegram-бот (ДЗ-2): уведомления есть, рабочего места нет — ни фильтров, ни истории, ни очереди на подпись

Цель

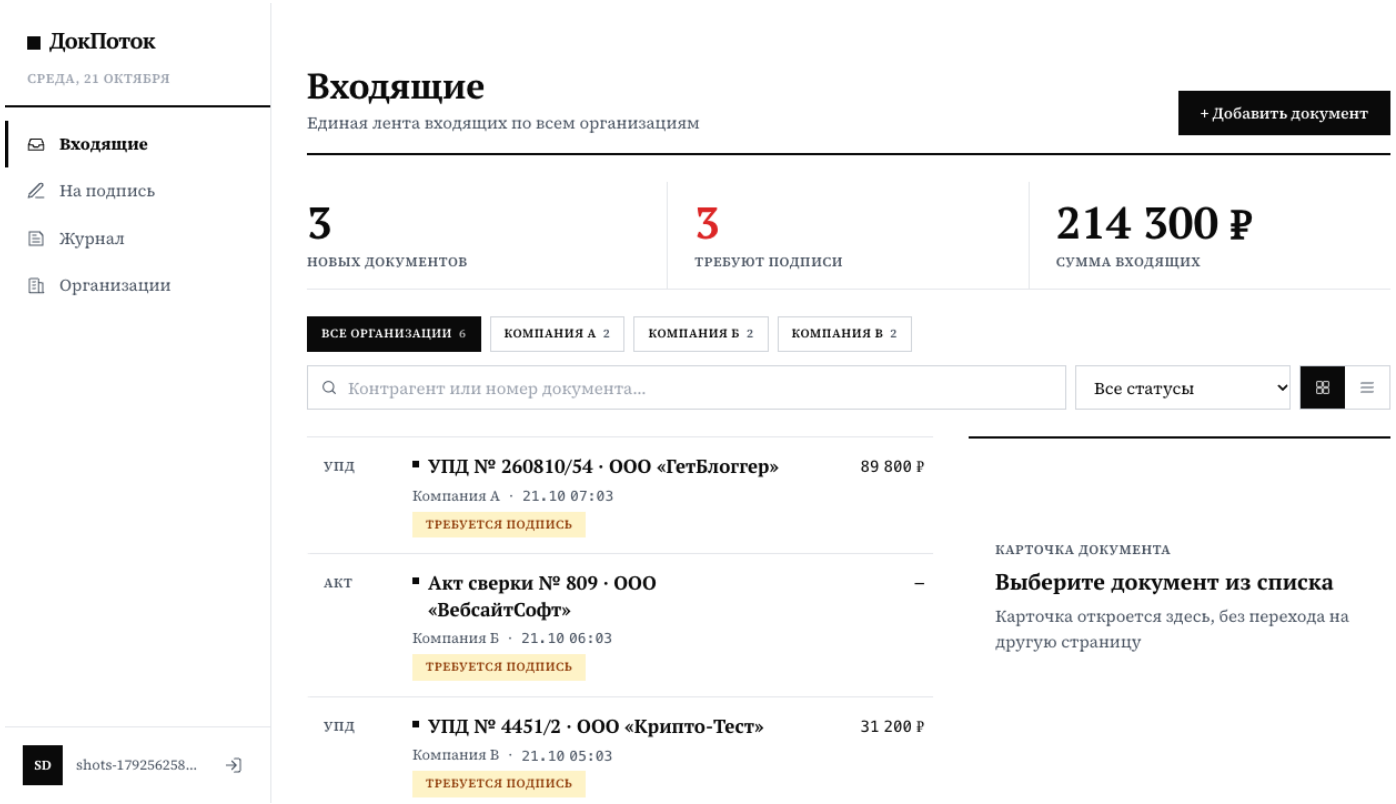
Один экран на все организации: лента, очередь «на подпись» с явным подтверждением, журнал, роли

Конкуренты: «несколько юрлиц» = переключатель, а не лента

Продукт	Несколько организаций	Роли и подписание
Контур.Диадок	Одна учётка → список организаций → переключение контекста. Единой ленты нет	Админ / редактор / читатель; маршруты согласования и «доп. подпись» — платные модули (10–230 тыс. ₽/год)
СБИС / Saby	Несколько компаний в аккаунте, тариф — на каждую; при отправке выбирается организация	Простое согласование в тарифе, маршруты — отдельная лицензия
1С-ЭДО (многофирменный учёт)	«Текущие дела ЭДО» — общий список по базе 1С; нужна 1С у каждого пользователя	Маршруты подписания несколькими сертификатами
DocuSign / PandaDoc	Switch Account / Workspaces (Enterprise) — изоляция, без общей входящей	Роли по аккаунту / workspace
Dext / Hubdoc (бухфирмы)	Ближайший паттерн: панель фирмы, очередь документов по каждому клиенту	Без юридически значимой подписи

Ниша: единая кросс-организационная лента + очередь «на подпись» по ролям + журнал. Главный риск — зависимость от API одного провайдера → абстракция EdoProvider в архитектуре. Анализ выполнен AI-агентом с веб-поиском (docs/competitors.md).

Решение: четыре экрана + вход



1 **Входящие**

Лента всех организаций, KPI, фильтры, поиск, карточка документа, добавление с PDF

2 **На подпись**

Очередь «требуется подпись» → диалог подтверждения → подписание по роли

3 **Журнал**

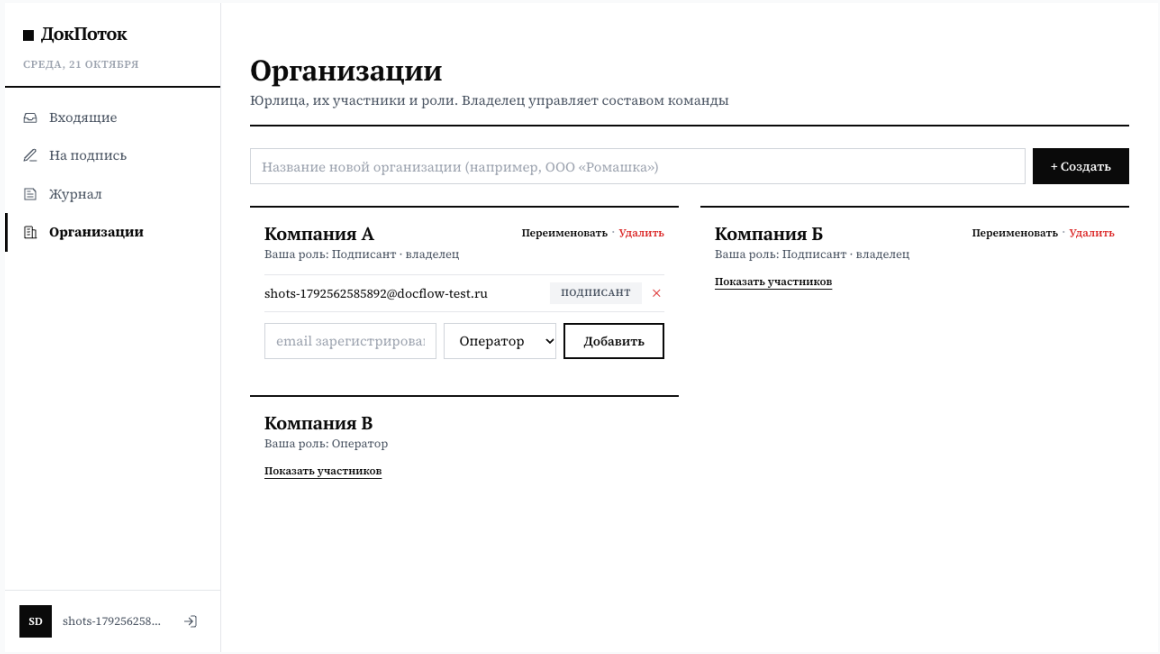
Append-only журнал: успехи и отказы с причиной

4 **Организации**

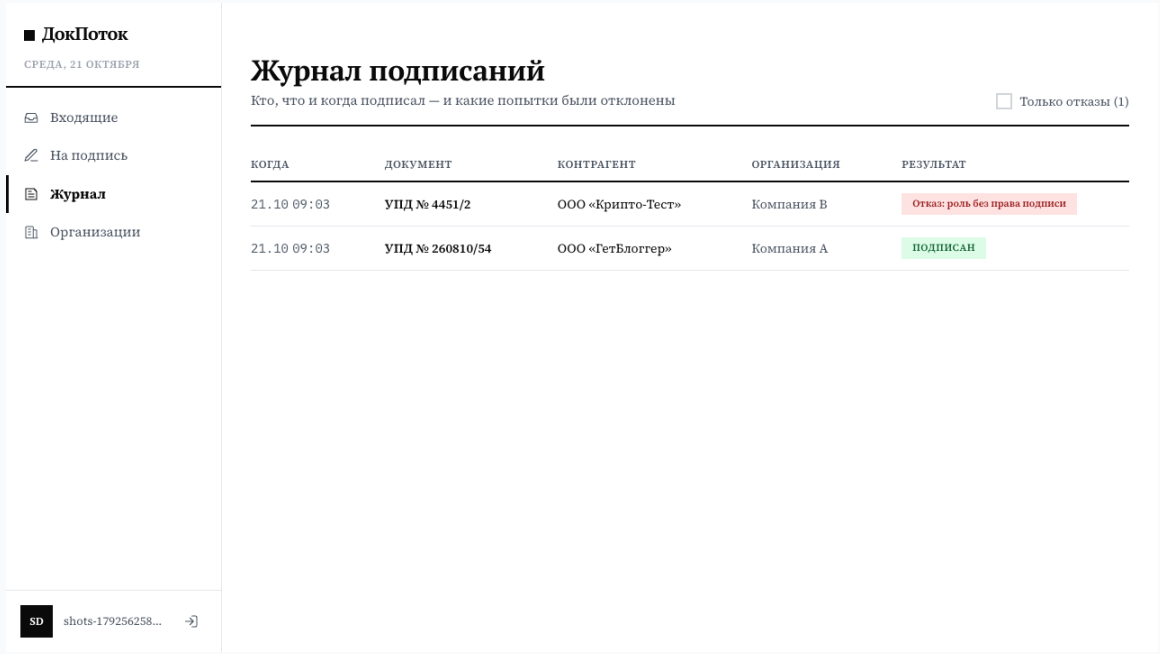
Создание, переименование, удаление; участники по email, роли

Адаптивно: ≥1024px — master-detail, на мобильных карточка открывается поверх списка. Каждый экран: loading / error / empty / success.

Организации и журнал подписаний



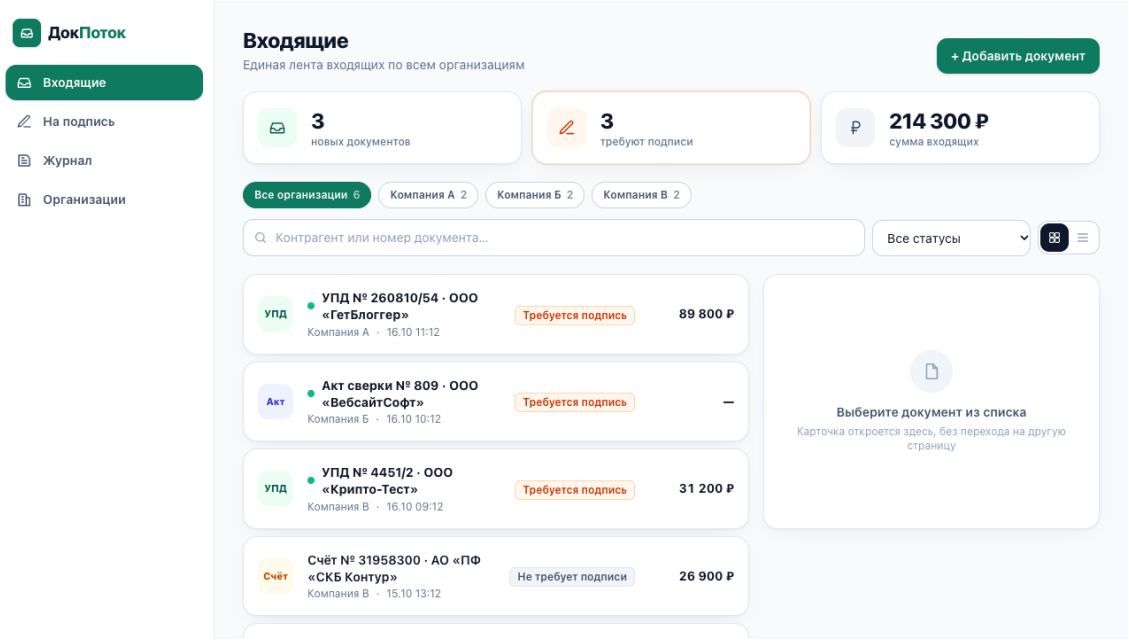
CRUD организаций; участники добавляются по email через RPC (auth.users клиенту недоступна); роль «оператор» без права подписи — для живой проверки политик



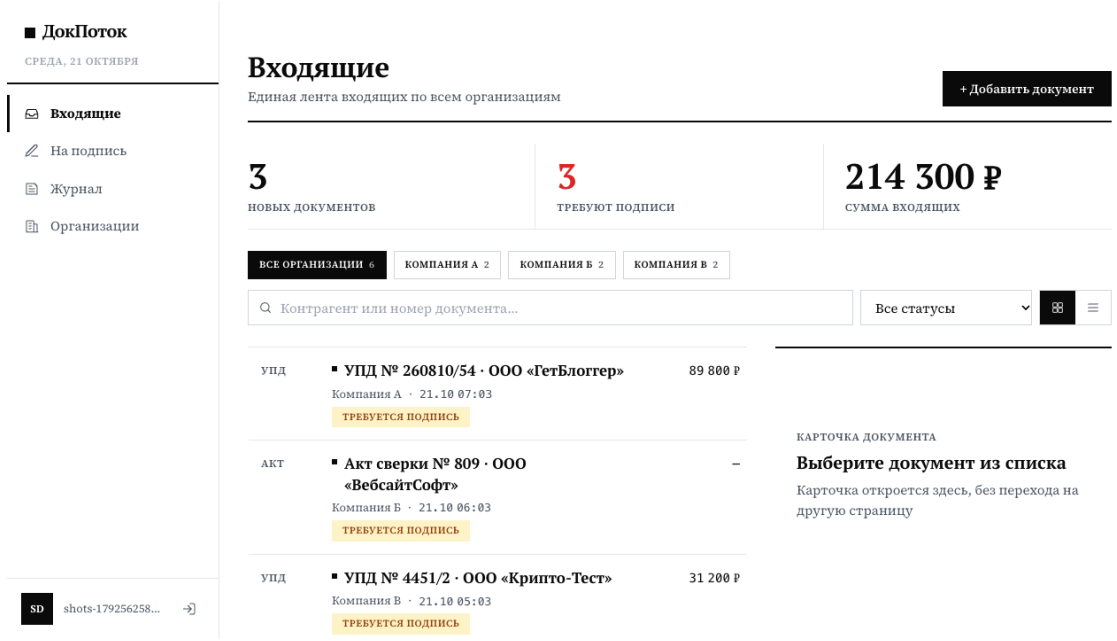
Журнал пишется только сервером (RPC), клиент не может его менять; отказы фиксируются с причиной — «роль без права подписи», «документ не требует подписи»

Интерфейс: готовая дизайн-система вместо «сгенерированной админки»

ДО — итерация 1



ПОСЛЕ — DESIGN.md (Broadsheet)

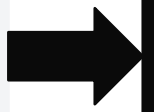


- Маркеры нейрослопа убраны: дефолтная палитра Tailwind, скругления и тени на всём, иконки в цветных квадратах, иллюстрации-заглушки, чек-лист «преимуществ» на входе
- DESIGN.md (формат Google Labs): YAML-токены + правила; взята готовая система Broadsheet (designmd.ai) — серифы, линейки вместо теней, радиус 0, один красный акцент; шрифты заменены на кириллические
- Токены спроецированы на Tailwind: стандартная палитра, rounded-* и shadow-* отключены на уровне темы; файл валидируется в CI. Скролл-эффекты: появление строк, полоса прогресса, липкий заголовок (с prefers-reduced-motion)

Архитектура

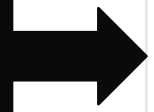
Frontend (GitHub Pages)

- React 19 + TypeScript strict
- React Router (HashRouter)
- Tailwind v4
- EdoProvider — граница с бэкендом
- мок-провайдер для тестов и демо



Supabase

- Auth: email + Google OAuth (PKCE)
- PostgREST: CRUD поверх 5 таблиц
- RPC: sign_document, add_member…
- Storage: приватный bucket PDF
- RLS + права уровня колонок



Эксплуатация

- GitHub Actions: audit → lint → prettier → tsc → DESIGN.md lint → 46 тестов → build → deploy
- Uptime-workflow + health RPC
- JSON-логи → client_logs
- Яндекс.Метрика: 14 событий

Ключевое решение: интерфейс EdoProvider. Фронтенд не знает, откуда данные — бэкенд подключался без изменения компонентов; тот же интерфейс закрывает риск зависимости от одного ЭДО-провайдера.

Нет прав администратора на машине → нет Docker/VPS → Supabase + GitHub Pages: ограничение стало фильтром решений.

Архитектура: почему именно так

Решение	Альтернатива	Почему
Supabase вместо своего сервера	Node/Express + Postgres в Docker на VPS	Нет прав администратора → нет Docker; нет VPS. Auth, RLS, REST — без собственного сервера
Безопасность в БД, а не в middleware	Проверки прав в API-слое	Без своего сервера единственная граница доверия — PostgreSQL: RLS, права колонок, security definer RPC
Интерфейс EdoProvider	Прямые вызовы supabase-js из компонентов	Мок → тесты и CI без секретов; бэкенд подключён без правки компонентов; ответ на риск зависимости от одного провайдера
GitHub Pages + HashRouter	Vercel/Netlify + BrowserRouter	Без второго аккаунта и токенов (OIDC); цена — нет SPA-fallback → hash-роутинг, решение задокументировано в App.tsx
Чистая логика вне React (src/lib)	Логика внутри компонентов	Правило 3 CLAUDE.md: фильтры, валидация, разбор OAuth-ошибок тестируются юнитами без DOM

Метод из ДЗ-1: сначала отсечь по жёстким ограничениям, потом сравнивать. Решения принимал человек; агент предлагал варианты и аргументы. Подробно: docs/agentic_engineering.md §1

Данные и безопасность

Таблица	Назначение
organizations	юрлица; created_by — владелец
org_members	роли: signer / operator / accountant (M:N с auth.users)
documents	входящие; file_path → Storage; check-constraints
sign_attempts	append-only журнал подписаний
client_logs	warn/error с фронтенда

6 миграций в supabase/migrations — проверены на живой базе API-тестами и e2e под свежим пользователем.

Подписание

Только через RPC с проверкой роли и статуса; у клиента отозваны права менять status — путь «мимо» физически невозможен

RLS везде

Видимость только своих организаций; анонимам — 401; is_member() как security definer против рекурсии

Аудит OWASP (AI)

12 находок, 11 исправлены: служебная функция была вызываема через RPC, TOCTOU в подписании, утечка сообщений сервера, CSP

Секреты

Ключи только в .env.local и GitHub Secrets; пароли тестовых учёток ротированы и убраны из репозитория

Что значит «подписание только через явное подтверждение»

1 · UI	Кнопка «Утвердить и подписать» не подписывает — открывает диалог с документом, контрагентом, суммой и подписантом. provider.sign() вызывает только «Подтвердить».	*CLAUDE.md, правило 8; тест «Отмена не меняет статус»*
2 · Данные	Обход UI невозможен: у роли authenticated отозван UPDATE на колонку status (права уровня колонок). Прямой PATCH status → 403.	*миграция 1; API-тест №4*
3 · Бизнес-правило	Единственный путь — RPC sign_document(): членство, роль signer, статус, блокировка строки, запись в append-only журнал (и отказов тоже).	*миграции 2, 3, 5; API-тесты №6–9*

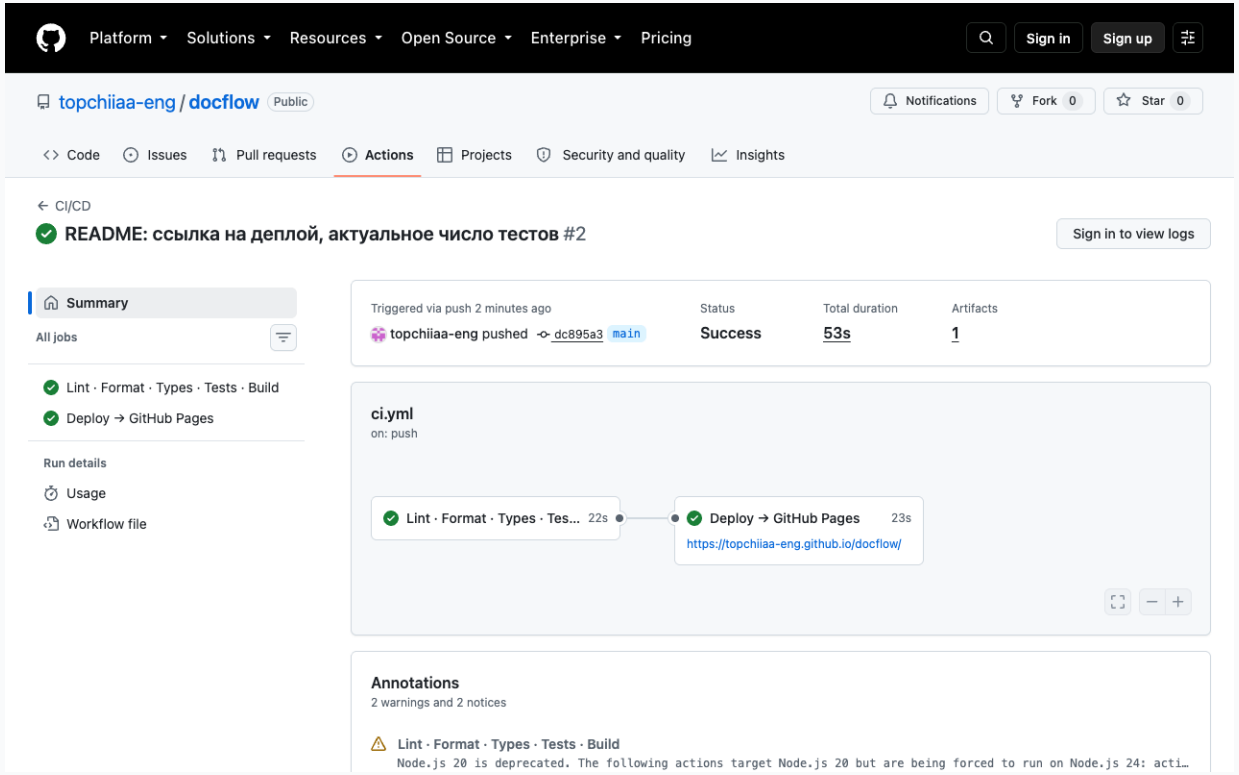
Происхождение: правило «никогда не подписывать автоматически» для AI-агента из ДЗ-2 (бот Диадока) перенесено с уровня инструкции на уровень архитектуры — подпись юридически значима.

AI-агент на каждом этапе

Этап	Как использован	Что дало
Планирование	Идея, анализ конкурентов (субагент + веб-поиск), 3 UI-концепции text-to-UI, 12 user stories	Подтверждённая ниша; концепция выбрана по главному сценарию
Архитектура	Схема БД и RLS по ТЗ; ТЗ проверено независимым агентом-«техлидом»	5 критичных пробелов в ТЗ найдены до первой строки кода
Frontend	Инициализация, компоненты по RTCF-промптам, тесты	Баг «бесконечный цикл» найден по скриншоту + консоли
Backend	Миграции, RPC, Storage-политики; разбор ошибок PostgreSQL	5 багов, не видимых локально, пойманы на живой базе
DevOps	Генерация CI/CD и uptime-workflow, аудит OWASP независимым агентом	Пайплайн дважды остановил деплой по делу
Оптимизация	Рекомендации линтера/агента: чанки, хуки, отмена устаревших ответов	Меньше лишних перерисовок; supabase-js кэшируется отдельно от кода

Правила проекта (CLAUDE.md) влияли на архитектуру, а не на форматирование: из «подписание только через диалог» вырос RPC с проверкой роли на уровне БД.

Качество: CI/CD, тесты, мониторинг



43

тестов: юниты, компоненты, провайдер

10

API-запросов к живой базе, 5 негативных

6

e2e-сценариев puppeteer на production

30 мин

интервал uptime-проверки + алерт

Пайплайн: npm audit → oxlint (0 предупреждений) → prettier → tsc → vitest → build → GitHub Pages → smoke-check. Дважды остановил деплой по делу: не включённый Pages и правки файлов в обход форматтера.

CI/CD: принципы из курса, а не дефолт агента

Принцип	Реализация	Что показал на практике
Fail-fast: дешёвое раньше дорогого	audit → lint → prettier → tsc → test → build	прогоны 4–5 упали на prettier за 20 с, не дойдя до сборки
Деплой зависит от качества	needs: quality; только main, не PR	«красное» не попало на сайт — job deploy пропущен
Секреты ≠ переменные; без PAT	ключи — Secrets; URL/ID — Variables; деплой по OIDC	публичное видно в логах, секретов нет; нечему утечь
Проверка после публикации	curl живого URL + job verify: Chrome на runner → вход → лента из Supabase → журнал	закрывает разрыв «деплой зелёный, приложение не работает» (кейсы 4–5)
Мониторинг как код	uptime.yml по cron: health RPC + сайт; падение = письмо	отработал в первом же запуске
Отмена устаревших, воспроизводимость	concurrency + cancel-in-progress; npm ci + lock + кэш; npm run ci локально	локальный и CI-прогоны совпадают

Сознательно нет: Docker-образов (нет сервера, который их запускал бы) и матрицы Node-версий (один runtime в production). Job verify добавлен по замечанию с защиты — см. docs/agentic_engineering.md §2.

Пять багов, которые не видели зелёные тесты

Бесконечный цикл загрузки

Дефолт пропа создавал новый провайдер на каждый рендер. Найдено по скриншоту + консоли браузера; 12 тестов молчали — они передавали стабильный проп.

Кириллица испорчена, миграция «Success»

pbcopy без UTF-8-локали. Найдено сравнением repr() строк из API. Урок: после миграции читать данные обратно.

Отказы не попадали в журнал

raise exception в plpgsql откатывал insert. RPC теперь возвращает {ok, error}. Урок: проверять побочные эффекты негативных сценариев.

RLS отклоняла создание организации

INSERT … RETURNING проверяется до AFTER-триггера членства. Нашёл только e2e на живой базе — мок и 43 теста не имеют RLS.

Новый пользователь — пустое приложение

Триггер владельца (миграция 4) × «не ронять регистрацию» (аудит): ошибка сидинга проглатывалась. Поймал скрипт скриншотов под свежим пользователем.

Общий вывод

«Зелёный прогон» ≠ «работает». Проверять данные и побочные эффекты другим инструментом — и сценарий «с нуля» после каждой миграции.

Что я поняла о разработке с AI

1

Агент сильнее всего, когда его проверяет другой агент

Лучшие находки — независимое ревью ТЗ, независимый аудит безопасности, API-тесты против живой базы

2

Правила влияют на архитектуру

CLAUDE.md «чистая логика без сети» → тестируемые модули; «подписание через диалог» → RPC на уровне БД

3

Жёсткие ограничения — лучший фильтр

Нет sudo → Supabase, GitHub Pages, системный Chrome для e2e — каждое решение оказалось проще альтернатив

4

Абстракция над провайдером окупилась дважды

Мок для тестов и CI без секретов + ответ на главный рыночный риск

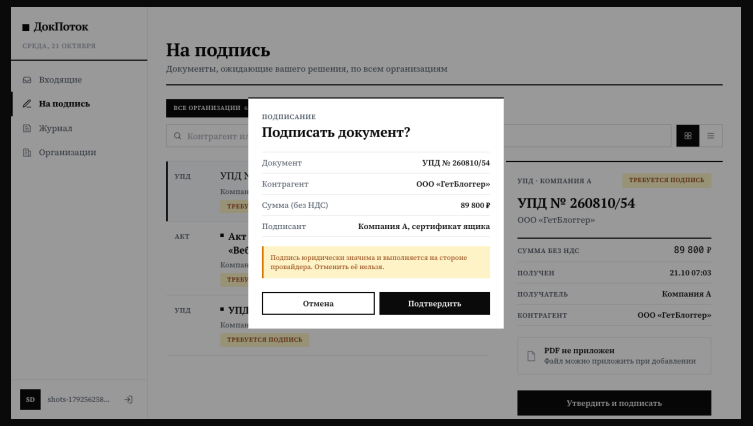
Agentic engineering ≠ вайбкодинг: где следы в репозитории

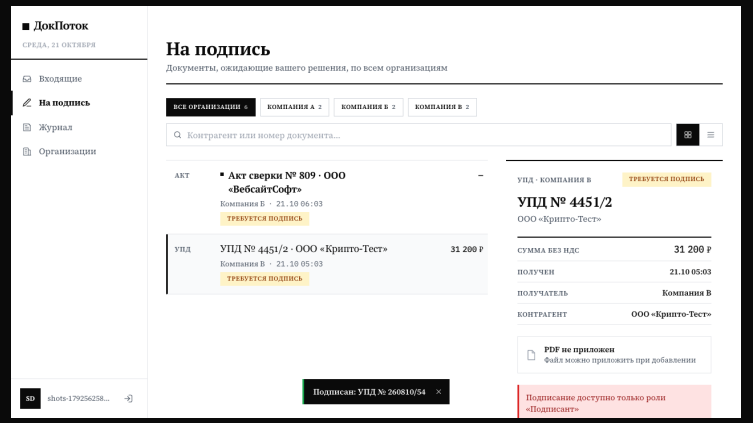
Признак	Вайбкодинг	В проекте	След
Правила до кода	агент пишет «как получится»	11 правил проекта; правило 7 → src/lib с юнитами, правило 8 → RPC в БД	CLAUDE.md
Агент проверяет агента	автор верит своему выводу	ревью ТЗ «техлидом» (5 пробелов), аудит OWASP (12 находок), конкуренты	security_audit.md, competitors.md
Верификация другим инструментом	«запустил — вроде работает»	46 тестов, 10 API-тестов к живой базе, 6 e2e на production, «свежий пользователь с нуля»	scripts/, src/**/*.test.*
Баги ловит проверка, не удача	замечает пользователь	5 багов прошли зелёные тесты — пойманы только верификацией; каждый с причиной	development_process.md, миграции 2, 5, 6
Журнал решений и история	один коммит «final»	шапка «почему» в каждой миграции; 25+ атомарных коммитов, Co-Authored-By	supabase/migrations, git log
Границы человека и честность	«AI сделал»	человек: архитектура, провайдеры, миграции в прод, приёмка, решения по аудиту; названо, что не сделано	agentic_engineering.md §4

Главное отличие: цикл замкнут на проверке, которую делает не тот, кто генерировал — другой агент, другой инструмент, другой сценарий. Честно: первые версии ci.yml и компонентов — чистая генерация; инженерная ценность появилась после прогонов, линтера и ревью.

Итоги

- 4 экрана, адаптивная вёрстка, формы с валидацией, 4 состояния экрана
- 5 связанных таблиц, полный CRUD, RLS, RPC, Storage, OAuth2, аналитика
- CI/CD → GitHub Pages, мониторинг, JSON-логи, аудит безопасности
- AI на всех этапах — задокументировано с промптами и результатами
- 46 тестов, 10 API-тестов, 6 e2e-сценариев; 5 багов найдены только на живой базе и исправлены





Приложение: topchiiaa-eng.github.io/docflow
Код и документация: github.com/topchiiaa-eng/docflow

Спасибо!
Вопросы?